Mini-Projetos Aplicados

Análise Topológica de Dados (TDA) – SME0231/SME5987

20 de outubro de 2026

Sumário

1 Projeto 1: Genômica Viral (Recombinação e Filodinâmica Topológica) 3

1.1 Visão Geral & Pergunta de Pesquisa . . . 3
1.2 Dados Públicos & Como Baixar . . . 3
1.3 Pipeline do Projeto . . . 3
1.4 Tópicos Avançados . . . 4
1.5 Código Modelo (Python) . . . 4

2 Projeto 2: Nuvens de Pontos 3D (Proteínas e Mudanças Conformacionais) 7

2.1 Visão Geral & Pergunta de Pesquisa . . . 7
2.2 Dados Públicos & Como Baixar . . . 7
2.3 Pipeline do Projeto . . . 7
2.4 Tópicos Avançados . . . 7
2.5 Código Modelo (Python) . . . 8

3 Projeto 3: Imagens 3D (Micro-CT e Microarquitetura Trabecular) 11

3.1 Visão Geral & Pergunta de Pesquisa . . . 11
3.2 Dados Públicos & Como Baixar . . . 11
3.3 Pipeline do Projeto . . . 11
3.4 Tópicos Avançados . . . 11
3.5 Código Modelo (Python) . . . 12

4 Projeto 4: Redes Complexas (Conectoma Funcional e Neurociência) 15

4.1 Visão Geral & Pergunta de Pesquisa . . . 15
4.2 Dados Públicos & Como Baixar . . . 15
4.3 Pipeline do Projeto . . . 15
4.4 Tópicos Avançados . . . 16

4.5 Código Modelo (Python) 16

5 Projeto 5: Séries Temporais (EEG e Crises Epilépticas via Takens) 20

5.1 Visão Geral & Pergunta de Pesquisa 20

5.2 Dados Públicos & Como Baixar 20

5.3 Pipeline do Projeto 20

5.4 Tópicos Avançados 21

5.5 Código Modelo (Python) 21

6 Ambiente Python e Boas Práticas 27

6.1 Ambiente Recomendado 27

6.2 Boas Práticas para os Projetos 27

7 Referências e Fontes de Dados 27

1 Projeto 1: Genômica Viral (Recombinação e Filodinâmica Topológica)

1.1 Visão Geral & Pergunta de Pesquisa

- **Contexto:** A evolução de vírus de RNA, como HIV, Influenza e Coronavírus, pode envolver recombinação ou *reassortment*, além de mutações. Esses processos podem produzir relações evolutivas que não são bem representadas por uma única árvore filogenética simples.
- **Pergunta de Pesquisa:** *"Há assinaturas topológicas persistentes em H_1 associadas a sequências recombinantes ou a regiões genômicas com recombinação conhecida, e essas assinaturas permanecem estáveis sob perturbações controladas dos dados?"*

Interpretação. Uma classe persistente em H_1 é compatível com estrutura não arbórea na geometria definida pela distância entre sequências, mas não constitui, isoladamente, prova de recombinação. A interpretação biológica deve ser confrontada com Formas Recombinantes Circulantes (CRFs), *breakpoints* conhecidos, simulações ou outro *ground truth*.

1.2 Dados Públicos & Como Baixar

- **Los Alamos HIV Sequence Database:** `https://www.hiv.lanl.gov/`. É uma fonte de alinhamentos curados de HIV, incluindo CRFs e regiões como *env*, *gag* e *pol*. Para o projeto, recomenda-se selecionar um conjunto pequeno e balanceado no portal e exportar o alinhamento em formato FASTA. [1, 2]
- **NCBI Entrez:** quando os identificadores de acesso (*accessions*) forem conhecidos, o módulo `Bio.Entrez` do Biopython permite baixar as sequências programaticamente. [3]
- **Nextstrain:** pode ser utilizado como fonte complementar de árvores, metadados e identificadores de sequências. Alguns conjuntos não redistribuem as sequências originais devido às restrições das fontes de dados. [4]

1.3 Pipeline do Projeto

1. **Fase 1 (Carga):** ler um arquivo FASTA previamente alinhado e verificar se todas as sequências possuem o mesmo comprimento.
2. **Fase 2 (Distância):** aplicar *complete deletion* nas colunas do alinhamento, retendo apenas posições A/C/G/T válidas em todas as sequências. Em seguida, calcular a distância de Hamming normalizada

$$d(s_i, s_j) = \frac{1}{m} \sum_{k=1}^{m} \mathbf{1}\{s_i(k) \neq s_j(k)\},$$

   onde m é o número de colunas válidas retidas.
3. **Fase 3 (Topologia):** construir o complexo de Vietoris–Rips a partir da matriz de distâncias. Para estudar H_1, o complexo deve ser criado pelo menos até dimensão 2, pois os triângulos são necessários para preencher e eliminar ciclos.

Para conjuntos didáticos pequenos, pode-se construir a filtração até o maior valor observado da matriz de distâncias. Caso um valor menor de `max_edge_length` seja utilizado por razões computacionais, classes persistentes que alcancem esse limite devem ser tratadas como censuradas. [5]

4. **Fase 4 (Análise):** extrair os intervalos persistentes de H_1 e comparar estatísticas como persistência máxima, persistência total ou representações vetoriais entre grupos, regiões genômicas ou simulações.
5. **Fase 5 (Validação Biológica):** comparar os sinais topológicos com CRFs ou *breakpoints* conhecidos e reportar medidas de desempenho ou efeitos estatísticos.

1.4 Tópicos Avançados

- **Estabilidade & Bottleneck:** aplicar mutações sintéticas com taxa crescente e medir a distância de *bottleneck* entre diagramas. Um resultado clássico de estabilidade possui a forma
$$d_B\big(D(f), D(g)\big) \leq \|f - g\|_\infty,$$
sob hipóteses apropriadas sobre as funções de filtração. [6]
- **Machine Learning:** deslocar uma janela ao longo do genoma, extrair *persistence images*, *persistence landscapes*, curvas de Betti ou entropia de persistência e treinar um classificador utilizando regiões com *breakpoints* conhecidos como referência. A divisão treino/teste deve evitar vazamento entre sequências fortemente aparentadas.
- **Kepler Mapper:** construir o Mapper sobre descritores das sequências utilizando como lente, por exemplo, ano de isolamento, subtipo ou uma componente de redução dimensional. O grafo deve ser interpretado como ferramenta exploratória.

1.5 Código Modelo (Python)

```python
from pathlib import Path
import numpy as np
import gudhi
from Bio import Entrez, SeqIO

VALID = set("ACGT")


def fetch_ncbi_fasta(accessions, out_fasta, email):
    """Baixa sequencias por accession; alinhe-as antes do Hamming."""
    Entrez.email = email

    with Entrez.efetch(
        db="nuccore",
        id=",".join(accessions),
        rettype="fasta",
        retmode="text",
    ) as handle:
        Path(out_fasta).write_text(
            handle.read(),
            encoding="utf-8",
        )

    return Path(out_fasta)
```

```
25
26
27  def load_aligned_fasta(path):
28      records = list(SeqIO.parse(path, "fasta"))
29
30      if len(records) < 3:
31          raise ValueError(
32              "Use pelo menos 3 sequencias alinhadas."
33          )
34
35      lengths = {len(r.seq) for r in records}
36
37      if len(lengths) != 1:
38          raise ValueError(
39              "As sequencias precisam estar alinhadas "
40              "e ter o mesmo comprimento."
41          )
42
43      return records
44
45
46  def distance_matrix(records):
47      # Complete deletion:
48      # mesmas colunas A/C/G/T para todos os pares.
49
50      seqs = np.array(
51          [list(str(r.seq).upper()) for r in records]
52      )
53
54      valid_cols = np.all(
55          np.isin(seqs, list(VALID)),
56          axis=0,
57      )
58
59      seqs = seqs[:, valid_cols]
60
61      if seqs.shape[1] == 0:
62          raise ValueError(
63              "Nenhuma coluna A/C/G/T valida "
64              "em todas as sequencias."
65          )
66
67      n = len(records)
68      D = np.zeros((n, n), dtype=float)
69
70      for i in range(n):
71          for j in range(i + 1, n):
72              D[i, j] = D[j, i] = np.mean(
73                  seqs[i] != seqs[j]
74              )
75
76      return D, int(valid_cols.sum())
77
78
79  def h1_from_distance_matrix(
80      D,
81      max_edge_length=None,
82  ):
83      D = np.asarray(D, dtype=float)
```

```
    upper = D[
        np.triu_indices_from(D, k=1)
    ]

    if not np.isfinite(upper).all():
        raise ValueError(
            "A matriz de distancias contem NaN/inf."
        )

    full_cutoff = float(np.max(upper))

    cutoff = (
        full_cutoff
        if max_edge_length is None
        else float(max_edge_length)
    )

    rips = gudhi.RipsComplex(
        distance_matrix=D,
        max_edge_length=cutoff,
    )

    st = rips.create_simplex_tree(
        max_dimension=2
    )

    st.compute_persistence()

    h1 = st.persistence_intervals_in_dimension(1)

    return h1, cutoff


# Exemplo:
#
# records = load_aligned_fasta(
#     "lanl_env_aligned.fasta"
# )
#
# D, n_sites = distance_matrix(records)
#
# h1, cutoff = h1_from_distance_matrix(D)
#
# Fetch alternativo pelo NCBI:
#
# fetch_ncbi_fasta(
#     ["U54771.1"],
#     "sequencias.fasta",
#     "seu@email.edu",
# )
```

Listing 1: FASTA alinhado, Hamming normalizada e persistência em H1

2 Projeto 2: Nuvens de Pontos 3D (Proteínas e Mudanças Conformacionais)

2.1 Visão Geral & Pergunta de Pesquisa

- **Contexto:** A função de uma proteína depende de sua geometria tridimensional e de mudanças conformacionais. A homologia persistente pode resumir características geométricas de uma nuvem atômica em diferentes escalas.
- **Pergunta de Pesquisa:** *"Como as assinaturas topológicas em H_1 e H_2 de uma mesma proteína mudam entre conformações não ligada e ligada, e essas assinaturas permitem discriminar estados conformacionais ou famílias estruturais em um conjunto maior?"*

Interpretação. Utilizando apenas átomos C_α, H_1 e H_2 descrevem a geometria do esqueleto proteico em escala grossa. Uma classe de H_2 não deve ser automaticamente interpretada como uma bolsa de ligação. Para estudar bolsos físicos, uma extensão do projeto pode incluir átomos pesados, superfícies moleculares e validação estrutural.

2.2 Dados Públicos & Como Baixar

- **RCSB Protein Data Bank:** `https://www.rcsb.org/`. O formato PDBx/mmCIF pode ser utilizado para download programático das estruturas. O RCSB disponibiliza arquivos por URLs como `https://files.rcsb.org/download/4AKE.cif`. [7]
- **Par de demonstração:** `4AKE` corresponde à adenilato-quinase de *Escherichia coli* em conformação não ligada, enquanto `1AKE` contém a mesma enzima complexada com AP5A. [8, 9]
- Para a comparação didática, será utilizada a cadeia A das duas estruturas.

2.3 Pipeline do Projeto

1. **Fase 1 (Fetch):** baixar `4AKE.cif` e `1AKE.cif` diretamente do RCSB.
2. **Fase 2 (Carga):** ler os arquivos PDBx/mmCIF com `Bio.PDB.MMCIFParser` e extrair as coordenadas (x, y, z) dos átomos C_α da cadeia A.
3. **Fase 3 (Topologia):** construir um **Alpha Complex** em $\mathbb{R}^3$. Para nuvens de pontos euclidianas de baixa dimensão, essa construção é particularmente apropriada. [10]
4. **Fase 4 (Análise):** extrair intervalos de persistência em H_1 e H_2 e comparar quantidade de classes, persistência máxima, persistência total e distâncias entre diagramas.
5. **Fase 5 (Interpretação):** relacionar as diferenças topológicas às mudanças conformacionais observadas entre as duas estruturas.

2.4 Tópicos Avançados

- **Estabilidade & Bottleneck:** calcular

$$d_B(D_{4\mathrm{AKE}}, D_{1\mathrm{AKE}})$$

em H_1 e H_2. No Alpha Complex do GUDHI, os valores de filtração são expressos em termos de raios ao quadrado, portanto comparações devem utilizar a mesma convenção. [10, 6]

- **Machine Learning:** para um conjunto maior de proteínas, transformar os diagramas em *persistence images* e treinar um classificador, como uma SVM, para distinguir estados conformacionais ou famílias estruturais.
- **Kepler Mapper:** utilizar excentricidade ou distância ao centro de massa como lente sobre os resíduos e comparar os agrupamentos obtidos com domínios estruturais conhecidos.

2.5 Código Modelo (Python)

```python
from pathlib import Path
import requests
import numpy as np
import gudhi
from Bio.PDB import MMCIFParser

RCSB = (
    "https://files.rcsb.org/download/"
    "{pdb_id}.cif"
)


def fetch_cif(
    pdb_id,
    out_dir="pdb_data",
):
    pdb_id = pdb_id.upper()

    out_dir = Path(out_dir)

    out_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

    path = out_dir / f"{pdb_id}.cif"

    if not path.exists():
        response = requests.get(
            RCSB.format(
                pdb_id=pdb_id
            ),
            timeout=60,
        )

        response.raise_for_status()

        path.write_bytes(
            response.content
        )

    return path

```

```
44
45  def ca_coordinates(
46      cif_path,
47      chain_id="A",
48  ):
49      parser = MMCIFParser(
50          QUIET=True
51      )
52
53      structure = parser.get_structure(
54          "protein",
55          str(cif_path),
56      )
57
58      model = next(
59          structure.get_models()
60      )
61
62      if chain_id not in model:
63          raise ValueError(
64              f"Cadeia {chain_id!r} "
65              "nao encontrada."
66          )
67
68      chain = model[chain_id]
69
70      coords = [
71          res["CA"].coord
72          for res in chain
73          if res.id[0] == " "
74          and "CA" in res
75      ]
76
77      coords = np.asarray(
78          coords,
79          dtype=float,
80      )
81
82      if coords.shape[0] < 4:
83          raise ValueError(
84              "Poucos atomos C-alpha."
85          )
86
87      return coords
88
89
90  def alpha_persistence(coords):
91      alpha = gudhi.AlphaComplex(
92          points=coords
93      )
94
95      st = alpha.create_simplex_tree()
96
97      st.compute_persistence()
98
99      return {
100         1: st.persistence_intervals_in_dimension(1),
101         2: st.persistence_intervals_in_dimension(2),
102     }
```

```
103
104
105 # Exemplo:
106 #
107 # apo = alpha_persistence(
108 #     ca_coordinates(
109 #         fetch_cif("4AKE")
110 #     )
111 # )
112 #
113 # holo = alpha_persistence(
114 #     ca_coordinates(
115 #         fetch_cif("1AKE")
116 #     )
117 # )
118 #
119 # dB_H2 = gudhi.bottleneck_distance(
120 #     apo[2],
121 #     holo[2],
122 # )
```

Listing 2: Download RCSB, coordenadas C-alpha e Alpha Complex

3 Projeto 3: Imagens 3D (Micro-CT e Microarquitetura Trabecular)

3.1 Visão Geral & Pergunta de Pesquisa

- **Contexto:** A microarquitetura do osso trabecular forma uma estrutura tridimensional complexa, cuja conectividade, espessura, separação e porosidade podem ser analisadas por métodos geométricos e topológicos.
- **Pergunta de Pesquisa:** *"A homologia persistente de uma filtração baseada em distância assinada consegue quantificar aspectos da conectividade e da porosidade trabecular e produzir descritores associados a medidas tradicionais como espessura trabecular (Tb.Th) e separação trabecular (Tb.Sp)?"*

3.2 Dados Públicos & Como Baixar

- **Zenodo/ORMIR – DOI 10.5281/zenodo.11061947:** coleção pública de amostras de osso trabecular segmentadas a partir de micro-CT. Os volumes possuem dimensão de $100 \times 100 \times 100$ voxels e voxel isotrópico de $17{,}59\,\mu$m. A coleção também contém transformadas de distância utilizadas no cálculo de Tb.Th e Tb.Sp. [11]
- **Arquivo para demonstração:** `BMLPL_009_BML_17_SEG_SUB.nii`. O código abaixo baixa diretamente o arquivo NIfTI e realiza sua leitura com `nibabel`.

3.3 Pipeline do Projeto

1. **Fase 1 (Fetch/Carga):** baixar o volume NIfTI segmentado e convertê-lo em uma máscara binária.
2. **Fase 2 (Filtração):** construir uma *signed Euclidean distance transform*, atribuindo valores negativos no interior do osso e positivos no espaço poroso.
   Uma possível definição é

   $$\phi(x) = d(x, \text{osso}) - d(x, \text{poro}),$$

   com o sinal ajustado de acordo com a convenção escolhida.
3. **Fase 3 (Topologia):** construir um `CubicalComplex` diretamente a partir do array tridimensional contendo os valores da filtração. [12]
4. **Fase 4 (Análise):** extrair os diagramas em H_0, H_1 e H_2. Em três dimensões, tanto a rede trabecular quanto o espaço poroso podem contribuir para diferentes classes topológicas.
5. **Fase 5 (Validação):** relacionar descritores topológicos com medidas clássicas como Tb.Th e Tb.Sp.

3.4 Tópicos Avançados

- **Estabilidade & Bottleneck:** introduzir perturbações de segmentação ou ruído, recomputar a filtração e medir a distância de bottleneck entre os diagramas obtidos.

- **Machine Learning:** utilizar *persistence images*, curvas de Betti ou estatísticas derivadas dos diagramas para regressão de Tb.Th e Tb.Sp.
- **Kepler Mapper:** representar cada volume por um vetor de descritores topológicos e/ou radiômicos e utilizar PCA ou UMAP como lente para investigar diferentes padrões de microarquitetura.

3.5 Código Modelo (Python)

```python
from pathlib import Path
import requests
import numpy as np
import nibabel as nib
import gudhi

from scipy.ndimage import (
    distance_transform_edt,
)

ZENODO_RECORD = "11061947"

FILENAME = (
    "BMLPL_009_BML_17_"
    "SEG_SUB.nii"
)

URL = (
    "https://zenodo.org/records/"
    f"{ZENODO_RECORD}/files/"
    f"{FILENAME}?download=1"
)


def fetch_bone_volume(
    out_dir="bone_data",
):
    out_dir = Path(out_dir)

    out_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

    path = out_dir / FILENAME

    if not path.exists():

        with requests.get(
            URL,
            stream=True,
            timeout=120,
        ) as response:

            response.raise_for_status()

            with path.open("wb") as f:

                for chunk in response.iter_content(
```

```
                    chunk_size=1024 * 1024
                ):
                    if chunk:
                        f.write(chunk)

    return path


def load_binary_mask(path):
    img = nib.load(str(path))

    data = np.asarray(
        img.get_fdata(),
        dtype=np.float32,
    )

    mask = data > 0.5

    spacing = tuple(
        float(v)
        for v in img.header.get_zooms()[:3]
    )

    return mask, spacing


def signed_distance_filtration(
    mask,
    sampling=(1.0, 1.0, 1.0),
):
    # Negativo no osso,
    # positivo no espaco poroso.

    d_bone = distance_transform_edt(
        mask,
        sampling=sampling,
    )

    d_pore = distance_transform_edt(
        ~mask,
        sampling=sampling,
    )

    return d_pore - d_bone


def cubical_persistence(
    filtration,
    stride=2,
):
    # stride=2 pode ser usado
    # para testes mais rapidos.

    arr = filtration[
        ::stride,
        ::stride,
        ::stride
    ]

```

```
    cubical = gudhi.CubicalComplex(
        top_dimensional_cells=arr
    )

    cubical.compute_persistence()

    return {
        0: cubical.persistence_intervals_in_dimension(0),
        1: cubical.persistence_intervals_in_dimension(1),
        2: cubical.persistence_intervals_in_dimension(2),
    }


# Exemplo:
#
# path = fetch_bone_volume()
#
# mask, spacing = load_binary_mask(path)
#
# phi = signed_distance_filtration(
#     mask,
#     sampling=spacing,
# )
#
# diagrams = cubical_persistence(
#     phi,
#     stride=2,
# )
```

Listing 3: Download Zenodo, NIfTI, distância assinada e Cubical Complex

4 Projeto 4: Redes Complexas (Conectoma Funcional e Neurociência)

4.1 Visão Geral & Pergunta de Pesquisa

- **Contexto:** O ABIDE reúne dados de neuroimagem de participantes com Transtorno do Espectro Autista (TEA) e controles, incluindo derivados do *Preprocessed Connectomes Project* (PCP). Redes funcionais podem ser construídas a partir de séries temporais regionais.
- **Pergunta de Pesquisa:** *"Existem diferenças reproduzíveis na persistência de H_1 dos conectomas funcionais entre participantes TEA e controles, após considerar o efeito de sítio e avaliar a generalização fora da amostra?"*

Escopo. O projeto utiliza um conjunto de dados de pesquisa para investigar associações topológicas entre grupos. Os resultados não devem ser interpretados como ferramenta de diagnóstico clínico individual.

4.2 Dados Públicos & Como Baixar

- **ABIDE Preprocessed/PCP via Nilearn:** a função `fetch_abide_pcp()` permite acessar dados derivados do ABIDE.
  O derivado `rois_cc200` contém séries temporais regionais baseadas no atlas Craddock 200. A função também retorna informações fenotípicas dos participantes. [13]
- Na codificação utilizada pela base, `DX_GROUP=1` corresponde ao grupo TEA e `DX_GROUP=2` ao grupo controle. [13]
- Para uma execução inicial, pode-se utilizar uma amostra de 20–40 participantes antes de ampliar a análise.

4.3 Pipeline do Projeto

1. **Fase 1 (Fetch):** baixar as séries `rois_cc200` por meio do Nilearn.
2. **Fase 2 (Conectividade):** para cada participante, calcular a matriz de correlação amostral de Pearson entre as séries temporais das regiões.
3. **Fase 3 (Distância):** converter a correlação em distância cordal:

$$d(i,j) = \sqrt{2(1 - r_{ij})}.$$

   Para vetores centrados e normalizados, essa transformação corresponde a uma distância euclidiana entre pontos sobre uma esfera unitária.
4. **Fase 4 (Topologia):** construir o complexo de Vietoris–Rips a partir da matriz de distâncias e extrair H_1.
   Para conjuntos de tamanho moderado, a filtração pode ser construída até a maior distância observada. Quando um corte menor for utilizado para reduzir o custo computacional, classes que atingirem esse limite devem ser interpretadas como censuradas. [5]

5. **Fase 5 (Comparação):** comparar as distribuições dos descritores topológicos entre os grupos considerando fatores potencialmente confundidores como sítio de aquisição, idade e sexo.

4.4 Tópicos Avançados

- **Estatística:** definir previamente uma estatística de comparação entre grupos e utilizar testes de permutação para estimar sua significância.
- **Machine Learning:** transformar diagramas em *persistence images*, *landscapes* ou outras representações vetoriais e utilizar classificadores como SVM.
  A validação pode utilizar `GroupKFold` ou uma estratégia *Leave-One-Site-Out* baseada em `SITE_ID`.
- **Kepler Mapper:** utilizar variáveis clínicas, como medidas ADOS quando disponíveis, como lentes para análise exploratória da população.

4.5 Código Modelo (Python)

```python
from pathlib import Path
import numpy as np
import gudhi

from nilearn.datasets import (
    fetch_abide_pcp,
)


def fetch_abide_timeseries(
    n_subjects=20,
    data_dir="abide_data",
):
    abide = fetch_abide_pcp(
        data_dir=Path(data_dir),
        n_subjects=n_subjects,
        pipeline="cpac",
        derivatives=["rois_cc200"],
        quality_checked=True,
        verbose=1,
    )

    def _load_roi(item):

        if isinstance(
            item,
            (str, Path),
        ):
            return np.loadtxt(item)

        return np.asarray(
            item,
            dtype=float,
        )

    time_series = [
        _load_roi(item)
```

```
        for item in abide.rois_cc200
    ]

    phen = (
        abide.phenotypic
        .reset_index(drop=True)
    )

    if len(time_series) != len(phen):
        raise RuntimeError(
            "Numero de series e linhas "
            "fenotipicas nao coincide."
        )

    return time_series, phen


def correlation_to_metric(corr):
    corr = np.clip(
        np.asarray(
            corr,
            dtype=float,
        ),
        -1.0,
        1.0,
    )

    D = np.sqrt(
        np.maximum(
            0.0,
            2.0 * (1.0 - corr),
        )
    )

    np.fill_diagonal(
        D,
        0.0,
    )

    return D


def subject_h1(
    ts,
    max_edge_length=None,
):
    ts = np.asarray(
        ts,
        dtype=float,
    )

    if (
        ts.ndim != 2
        or not np.isfinite(ts).all()
    ):
        raise ValueError(
            "A serie regional deve "
            "ser uma matriz 2D finita."
        )
```

```
97
98      # Pearson amostral entre colunas:
99      # cada coluna representa uma regiao.
100
101     corr = np.corrcoef(
102         ts,
103         rowvar=False,
104     )
105
106     if not np.isfinite(corr).all():
107         raise ValueError(
108             "Correlacao indefinida; "
109             "verifique regioes constantes."
110         )
111
112     D = correlation_to_metric(corr)
113
114     upper = D[
115         np.triu_indices_from(D, k=1)
116     ]
117
118     full_cutoff = float(
119         np.max(upper)
120     )
121
122     cutoff = (
123         full_cutoff
124         if max_edge_length is None
125         else float(max_edge_length)
126     )
127
128     rips = gudhi.RipsComplex(
129         distance_matrix=D,
130         max_edge_length=cutoff,
131     )
132
133     st = rips.create_simplex_tree(
134         max_dimension=2
135     )
136
137     st.compute_persistence()
138
139     return (
140         st.persistence_intervals_in_dimension(1),
141         cutoff,
142     )
143
144
145 # Exemplo:
146 #
147 # series, phen = fetch_abide_timeseries(
148 #     n_subjects=20
149 # )
150 #
151 # diagrams = [
152 #     subject_h1(ts)[0]
153 #     for ts in series
154 # ]
155 #
```

```
# y = (
#     phen["DX_GROUP"].to_numpy() == 1
# ).astype(int)
#
# y = 1 -> TEA
# y = 0 -> controle
```

Listing 4: Fetch ABIDE PCP, correlação, distância e H1

5 Projeto 5: Séries Temporais (EEG e Crises Epilépticas via Takens)

5.1 Visão Geral & Pergunta de Pesquisa

- **Contexto:** A imersão de Takens transforma uma série temporal escalar em uma nuvem de pontos no espaço de estados. Mudanças de periodicidade, organização temporal e sincronização podem alterar sua geometria e, consequentemente, sua homologia persistente.
- **Pergunta de Pesquisa:** *"A persistência de H_1 da reconstrução de Takens muda sistematicamente entre janelas interictais, pré-ictais e ictais no CHB-MIT, podendo funcionar como descritor para detecção e, de forma exploratória, alerta antecipado?"*

Interpretação. Uma crise epiléptica real não produz necessariamente um anel perfeito no espaço de Takens, assim como um EEG interictal pode apresentar estruturas persistentes em H_1. A hipótese deve ser testada empiricamente ao longo de diferentes janelas e pacientes.

5.2 Dados Públicos & Como Baixar

- **PhysioNet – CHB-MIT Scalp EEG Database:** `https://physionet.org/content/chbmit/1.0.0/`. A base contém EEG de longa duração em formato EDF, juntamente com arquivos de resumo que indicam os intervalos temporais das crises. [14]
- **Exemplo:** `chb01_03.edf`. O arquivo possui taxa de amostragem de 256 Hz e contém uma crise anotada entre 2996 s e 3036 s. O canal `FP1-F7` está presente na montagem. [15]
- O exemplo abaixo recorta uma região temporal próxima à crise, aplica um filtro passa-faixa, reamostra o sinal para 64 Hz e utiliza janelas curtas para a construção do Vietoris–Rips.

5.3 Pipeline do Projeto

1. **Fase 1 (Fetch/Carga):** baixar o EDF e realizar a leitura com MNE.
2. **Fase 2 (Pré-processamento):** escolher um canal, aplicar filtragem passa-faixa, reamostrar o sinal e padronizar cada janela antes da reconstrução.
3. **Fase 3 (Imersão de Takens):** para dimensão d e atraso τ, construir
$$v(t) = \big(x(t), x(t+\tau), x(t+2\tau), \ldots, x(t+(d-1)\tau)\big).$$
Para a demonstração pode-se utilizar $d = 3$ e um atraso pequeno. Em uma análise mais completa, d e τ podem ser selecionados utilizando autocorrelação, informação mútua, falsos vizinhos ou critérios relacionados.
4. **Fase 4 (Topologia):** construir um complexo de Vietoris–Rips para cada janela e extrair a persistência em H_1.
A subamostragem temporal reduz o número de pontos da nuvem. Quando a filtração for truncada antes da maior distância observada, classes persistentes que alcançam o limite devem ser interpretadas como censuradas.

5. **Fase 5 (Análise Temporal):** acompanhar estatísticas como persistência máxima, persistência total, entropia de persistência ou *landscapes* ao longo do tempo e compará-las às anotações das crises.

5.4 Tópicos Avançados

- **Estabilidade e transições:** calcular

$$d_B(D_t, D_{t-1})$$

  entre diagramas de janelas consecutivas. Valores elevados podem indicar mudanças na geometria do espaço reconstruído.
- **Machine Learning:** utilizar características topológicas de cada janela como entrada de modelos de classificação.
  A divisão treino/teste deve ser realizada por paciente sempre que o objetivo for avaliar generalização para novos indivíduos.
- **Previsão:** para investigar antecipação de crises, é necessário definir formalmente o horizonte pré-ictal, o período de exclusão, a taxa de falsos alarmes por hora e a estratégia de avaliação.
- **Kepler Mapper:** aplicar Mapper ao conjunto de janelas utilizando o tempo relativo ao início da crise ou descritores fisiológicos/topológicos como lente.

5.5 Código Modelo (Python)

```python
from pathlib import Path
import requests
import numpy as np
import mne
import gudhi

BASE = (
    "https://physionet.org/files/"
    "chbmit/1.0.0/chb01"
)

EDF = "chb01_03.edf"

SEIZURE = (
    2996.0,
    3036.0,
)


def fetch_edf(
    out_dir="chbmit_data",
):
    out_dir = Path(out_dir)

    out_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

```

```
    path = out_dir / EDF

    if not path.exists():

        with requests.get(
            f"{BASE}/{EDF}",
            stream=True,
            timeout=180,
        ) as response:

            response.raise_for_status()

            with path.open("wb") as f:

                for chunk in response.iter_content(
                    chunk_size=1024 * 1024
                ):
                    if chunk:
                        f.write(chunk)

    return path


def load_channel(
    path,
    channel="FP1-F7",
    tmin=2966.0,
    tmax=3046.0,
    fs=64,
):
    raw = mne.io.read_raw_edf(
        path,
        preload=False,
        verbose=False,
    )

    if channel not in raw.ch_names:
        raise ValueError(
            f"Canal {channel!r} nao encontrado; "
            f"exemplos: {raw.ch_names[:5]}"
        )

    raw = (
        raw.copy()
        .pick([channel])
        .crop(
            tmin=tmin,
            tmax=tmax,
        )
        .load_data()
    )

    raw.filter(
        1.0,
        30.0,
        verbose=False,
    )

    raw.resample(
```

```
        fs,
        verbose=False,
    )

    x = raw.get_data()[0]

    return (
        x,
        float(raw.info["sfreq"]),
    )


def takens_embedding(
    x,
    dim=3,
    delay=4,
    stride=4,
):
    x = np.asarray(
        x,
        dtype=float,
    )

    x = (
        x - x.mean()
    ) / (
        x.std() + 1e-12
    )

    n = (
        len(x)
        - (dim - 1) * delay
    )

    if n <= 0:
        raise ValueError(
            "Sinal curto demais para "
            "dim/delay escolhidos."
        )

    emb = np.column_stack(
        [
            x[
                i * delay:
                i * delay + n
            ]
            for i in range(dim)
        ]
    )

    return emb[::stride]


def h1_window(
    x,
    dim=3,
    delay=4,
    stride=4,
    max_edge_length=None,
```

```
):
    X = takens_embedding(
        x,
        dim=dim,
        delay=delay,
        stride=stride,
    )

    diff = (
        X[:, None, :]
        - X[None, :, :]
    )

    D = np.sqrt(
        np.sum(
            diff * diff,
            axis=-1,
        )
    )

    upper = D[
        np.triu_indices_from(
            D,
            k=1,
        )
    ]

    full_cutoff = float(
        np.max(upper)
    )

    cutoff = (
        full_cutoff
        if max_edge_length is None
        else float(max_edge_length)
    )

    rips = gudhi.RipsComplex(
        points=X,
        max_edge_length=cutoff,
    )

    st = rips.create_simplex_tree(
        max_dimension=2
    )

    st.compute_persistence()

    return (
        st.persistence_intervals_in_dimension(1),
        cutoff,
    )


def window_scores(
    x,
    fs,
    window_s=8.0,
    step_s=4.0,
```

```
):
    w = int(
        window_s * fs
    )

    step = int(
        step_s * fs
    )

    rows = []

    for start in range(
        0,
        len(x) - w + 1,
        step,
    ):
        seg = x[
            start:start + w
        ]

        h1, cutoff = h1_window(seg)

        finite = (
            h1[
                np.isfinite(
                    h1[:, 1]
                )
            ]
            if len(h1)
            else np.empty((0, 2))
        )

        max_persistence = (
            float(
                np.max(
                    finite[:, 1]
                    - finite[:, 0]
                )
            )
            if len(finite)
            else 0.0
        )

        rows.append(
            (
                start / fs,
                max_persistence,
                cutoff,
            )
        )

    return rows


# Exemplo:
#
# path = fetch_edf()
#
# x, fs = load_channel(path)
```

```
#
# scores = window_scores(
#     x,
#     fs,
# )
#
# Como o recorte comeca em 2966 s,
# a crise oficial de 2996-3036 s
# ocupa aproximadamente 30-70 s
# no sinal recortado.
```

Listing 5: Fetch CHB-MIT, MNE, Takens e H1 por janela

6 Ambiente Python e Boas Práticas

6.1 Ambiente Recomendado

Uma instalação adequada para executar os exemplos é:

```
python -m pip install numpy scipy pandas matplotlib requests \
    "biopython==1.88" "gudhi==3.13.0" nibabel \
    "nilearn==0.14.1" mne scikit-learn

# Extensoes opcionais:
python -m pip install kmapper umap-learn
```

6.2 Boas Práticas para os Projetos

- Registrar as versões de Python e dos pacotes utilizados, por exemplo com `pip freeze` ou `conda env export`.
- Guardar os identificadores dos dados utilizados: accessions, PDB IDs, nomes de arquivos, sujeitos e parâmetros de processamento.
- Fixar sementes aleatórias em procedimentos estocásticos como UMAP, divisões de dados e algoritmos de Machine Learning.
- Não escolher hiperparâmetros com base no desempenho observado no conjunto de teste.
- Reportar o tamanho da amostra, critérios de inclusão/exclusão e dados ausentes.
- Distinguir análises exploratórias de testes confirmatórios.
- Comparar descritores topológicos com pelo menos um *baseline* simples e relevante para o problema.
- Em Machine Learning, respeitar a estrutura de dependência dos dados ao realizar a divisão treino/teste: linhagem ou sequência, família proteica, amostra, sítio ou paciente, conforme o projeto.

7 Referências e Fontes de Dados

Referências

[1] Los Alamos HIV Sequence Database. *Sequence Alignment Help*. https://www.hiv.lanl.gov/content/sequence/NEWALIGN/help.html

[2] Los Alamos HIV Sequence Database. *HIV Circulating Recombinant Forms (CRFs)*. https://www.hiv.lanl.gov/components/sequence/HIV/crfdb/crfs.comp

[3] Biopython. *Bio.Entrez API documentation*. https://biopython.org/docs/latest/api/Bio.Entrez.html

[4] Nextstrain. *Download the data displayed through nextstrain.org*. https://docs.nextstrain.org/en/latest/guides/share/download-data.html

[5] GUDHI. *Rips Complex documentation*. https://gudhi.inria.fr/python/latest/rips_complex_user.html

[6] GUDHI. *Bottleneck Distance documentation*. https://gudhi.inria.fr/python/latest/bottleneck_distance_user.html

[7] RCSB Protein Data Bank. *File Download Services*. https://www.rcsb.org/docs/programmatic-access/file-download-services

[8] RCSB Protein Data Bank. *4AKE: Adenylate Kinase*. https://www.rcsb.org/structure/4AKE

[9] RCSB Protein Data Bank. *1AKE: Adenylate kinase complexed with AP5A*. https://www.rcsb.org/structure/1AKE

[10] GUDHI. *Delaunay and Alpha Complex documentation*. https://gudhi.inria.fr/python/latest/delaunay_complex_ref.html

[11] Kuczynski, M. *MicroCT Trabecular Bone Samples for Trabecular Thickness and Separation Measures*. Zenodo, 2024. DOI: 10.5281/zenodo.11061947. https://zenodo.org/records/11061947

[12] GUDHI. *Cubical Complex documentation*. https://gudhi.inria.fr/python/latest/cubical_complex_ref.html

[13] Nilearn. *nilearn.datasets.fetch_abide_pcp*. https://nilearn.github.io/stable/modules/generated/nilearn.datasets.fetch_abide_pcp.html

[14] PhysioNet. *CHB-MIT Scalp EEG Database v1.0.0*. https://physionet.org/content/chbmit/1.0.0/

[15] PhysioNet. *CHB-MIT Subject 01 Summary*. https://physionet.org/files/chbmit/1.0.0/chb01/chb01-summary.txt